

Aprendizaje Automático

Introducción a Deep Learning

Manuel Pegalajar Cuéllar¹

¹Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada

Contenido

- 1 Redes Neuronales y Deep Learning
- 2 Tensorflow
- 3 Introducción a Keras
- 4 Redes convolucionales
- 5 Autoencoders
- 6 Ejercicios y entrega

- 1 Redes Neuronales y Deep Learning
- 2 Tensorflow
- 3 Introducción a Keras
- 4 Redes convolucionales
- 5 Autoencoders
- 6 Ejercicios y entrega

Concepto (Redes Neuronales Artificiales)

Modelos matemáticos (parametrizados) inspirados en el funcionamiento del cerebro humano. El proceso de optimización de parámetros se denomina aprendizaje.

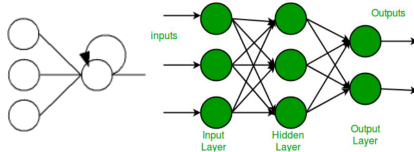
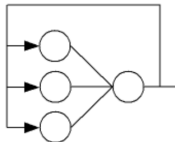
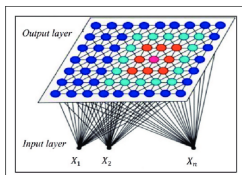
Representación gráfica: Grafo $G = \langle V, A \rangle$

- **V**: Conjunto de vértices $\{v_i\}$
- **A**: Conjunto de aristas (conexiones) $a = (v_i, v_j)$ **ponderadas** con pesos w_{ij}

Redes neuronales artificiales

Tipos de conexiones en redes neuronales artificiales:

- **Simétricas.**
- **Asimétricas**
- **Dirigidas hacia adelante**
- **Recurrentes**

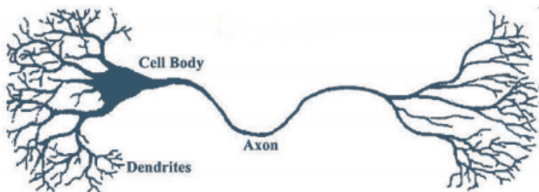


Redes neuronales artificiales

Concepto (Modelo computacional básico inspirado en el modelo biológico fundamental)

Una **neurona** recibe **impulsos eléctricos (entradas)** a través de las **dendritas**. Estos impulsos son **agregados (suma ponderada)** en el cuerpo de la neurona. Si la cantidad de pulsos eléctricos agregados supera un umbral (**umbral de activación**), que depende de la **plasticidad** de la neurona, esta transmitirá un impulso eléctrico (**salida**) por el **axón** a otras neuronas.

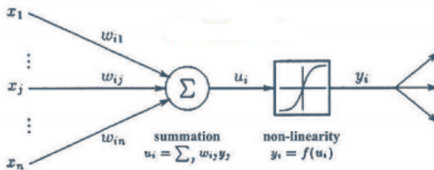
Cuando una neurona transmite un impulso por el axón, se dice que **la neurona se activa**.



Redes neuronales artificiales

Modelo computacional:

- $\{x_j\}_{j=1}^N$: N impulsos eléctricos (**entradas**) a una neurona.
- w_{ij} : Importancia (**ponderación**) de la entrada x_j para la neurona i .
- $f(u_i)$: Función para controlar la activación de la neurona i en función de la agregación de entradas u_i (**función de activación**).



Modelo computacional:

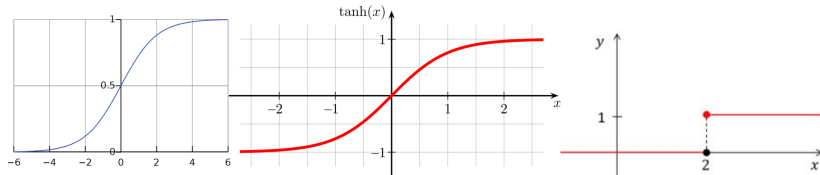
$$u_i = w_0 + \sum_{j=1}^N (w_{ij}x_j)$$

$$y_i = f(u_i)$$

El término w_0 es un **umbral de activación** de la neurona, más conocido en el área del aprendizaje automático como **bias**.

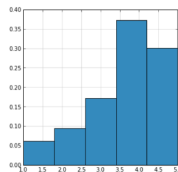
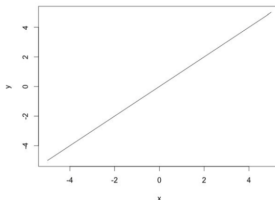
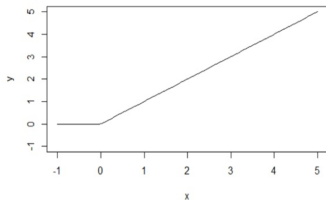
Funciones de activación comunes utilizadas en redes neuronales y Deep Learning:

- **Sigmoide:** $f(x) = \frac{1}{1+e^{-x}}$
- **Tangente Hiperbólica (tanh):** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- **Escalón:** $H(x) = \begin{cases} 1 & ; x \geq 0 \\ 0 & ; x < 0 \end{cases}$



Funciones de activación comunes utilizadas en redes neuronales y Deep Learning (continuación):

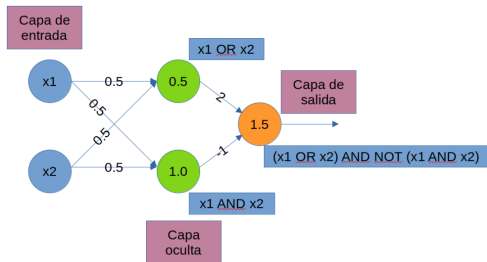
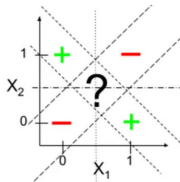
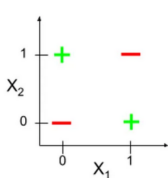
- **ReLU**: $f(x) = \max\{0, x\}$
- **Lineal**: $f(x) = x$
- **SoftMax** (distribución de probabilidad discreta):
$$\text{SoftMax}(u_i) = \frac{e^{u_i}}{\sum_j e^{u_j}}$$
- **etc.**



Redes neuronales artificiales

En una red neuronal, las neuronas las solemos agrupar en **capas**. Las **redes feedforward** son secuencias de capas de neuronas. Cada capa se encarga de hacer **una transformación a los datos de entrada a la capa**. Por ejemplo, la red neuronal siguiente resuelve el problema de clasificación del **XOR** con dos capas y función de activación $H(x)$:

Problema del XOR

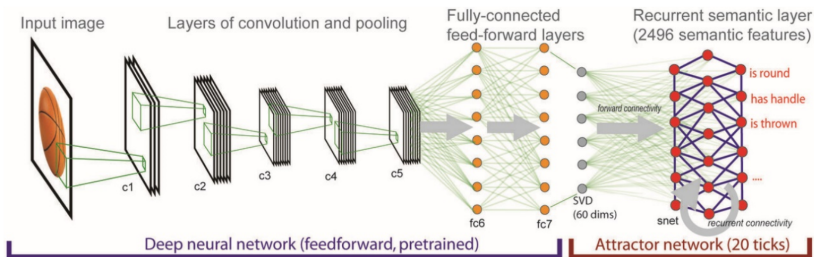


Deep Learning

Los modelos actuales de Deep Learning incluyen la integración de un elevado número de capas, cada una con un tipo de neuronas o estructura diferente. A nivel “oficial”, un modelo de Deep Learning se define:

Concepto (Deep Learning Models)

(...) computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction (LeCun et al., 2015)



Contenido

- 1 Redes Neuronales y Deep Learning
- 2 **Tensorflow**
- 3 Introducción a Keras
- 4 Redes convolucionales
- 5 Autoencoders
- 6 Ejercicios y entrega

Tensorflow es una herramienta de Google para cálculo científico. Está entre las más potentes a día de hoy debido a su integración con sistemas hardware para cálculos intensivo.

La **API** de alto nivel de **Tensorflow** para generación de modelos de Deep Learning se denomina **Keras**.

El aprendizaje de **Tensorflow** es relativamente sencillo a nivel básico:

<https://www.tensorflow.org/tutorials?hl=es-419>

Migración para usuarios de NumPy:

- Fácil adaptación.
- Muchas operaciones y atributos mantienen nombre.
- Tipo central en **NumPy**: Array Multidimensional (**ndarray**).
- Tipo central en **TensorFlow**: Array Multidimensional (**tensor**).
- Conversión de tipos entre **NumPy** y **Tensorflow** simple.

OJO: Un tensor es un array multidimensional CONSTANTE.

Tensores modificables: Tipo **Variable**. No se puede modificar con indexación del tipo $a[i] = 7.0$.

Tensores constantes vs. variables:

El constante no se puede modificar. El variable sólo con operaciones de TensorFlow (internamente).

```
1 import tensorflow as tf
2
3 # Tensor constante (no modificable)
4 a= tf.constant([1, 2, 3], shape=(3,), dtype=tf.float32)
5
6 # Tensor variable (modificable con metodos de TF)
7 b= tf.Variable([1, 2, 3], shape=(3,), dtype=tf.float32)
8 b.assign([3, 2, 1])
9
```


Transformación de arrays entre Numpy y Tensorflow:

```
1 import tensorflow as tf
2 import numpy as np
3
4 cn= np.array([1, 2, 3])
5
6 # De numpy a TF
7 ct= tf.convert_to_tensor(cn)
8
9 # de TF a numpy
10 cn2= ct.numpy()
11
```



Ejemplos de creación de tensores:

```
1  import tensorflow as tf
2
3  Z= tf.zeros(shape=(3, 2), dtype=tf.float32)
4
5  O= tf.ones(shape=(3), dtype=tf.float32)
6
7  E= tf.eye(num_rows=3)
8
```

Ejemplos de cambio de forma de tensores:

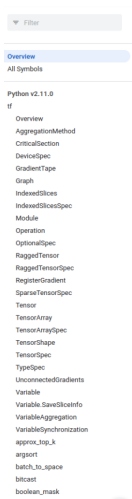
```
1 import tensorflow as tf
2
3 E= tf.eye(num_rows=2)
4
5 E_0= tf.reshape(E, shape=(1, 4)) # [[1, 0, 0, 1]]
6 E_0_sq= tf.squeeze(E_0) # [1, 0, 0, 1]
7
```

Algunas operaciones matemáticas:

- `tf.add`
- `tf.multiply`
- `tf.linalg.matmul`
- `tf.random.uniform`
- `tf.reduce_mean`
- `tf.reduce_sum`
- `etc., etc., etc., ...`



La API es muy extensa: https://www.tensorflow.org/api_docs/python/tf



Functions

`Assert(...)`: Asserts that the given condition is true.

`abs(...)`: Computes the absolute value of a tensor.

`acos(...)`: Computes acos of x element-wise.

`acosh(...)`: Computes inverse hyperbolic cosine of x element-wise.

`add(...)`: Returns $x + y$ element-wise.

`add_n(...)`: Returns the element-wise sum of a list of tensors.

`approx_top_k(...)`: Returns min/max k values and their indices of the input operand in an approximate manner.

`argmax(...)`: Returns the index with the largest value across axes of a tensor.

`argmin(...)`: Returns the index with the smallest value across axes of a tensor.

`argsort(...)`: Returns the indices of a tensor that give its sorted order along an axis.

`as_dtype(...)`: Converts the given `type_value` to a `DType`.

`as_string(...)`: Converts each entry in the given tensor to strings.

`asin(...)`: Computes the trigonometric inverse sine of x element-wise.

`asinh(...)`: Computes inverse hyperbolic sine of x element-wise.

`assert_equal(...)`: Assert the condition $x == y$ holds element-wise.

`assert_greater(...)`: Assert the condition $x > y$ holds element-wise.

`assert_less(...)`: Assert the condition $x < y$ holds element-wise.

`assert_rank(...)`: Assert that x has rank equal to `rank`.

`atan(...)`: Computes the trigonometric inverse tangent of x element-wise.

`atan2(...)`: Computes arctangent of y/x element-wise, respecting signs of the arguments.

`atanh(...)`: Computes inverse hyperbolic tangent of x element-wise.

`batch_to_space(...)`: BatchToSpace for N-D tensors of type T.

`bitcast(...)`: Bitcasts a tensor from one type to another without copying data.

Indexación con tensores:

Es similar (en ocasiones distinta) a NumPy. Se contemplan varias formas de indexación de tensores:

- Indexación con el **operador dos puntos** ":"
- Indexación **booleana**.
- División de tensor en partes uniformes con **tf.split**.
- Concatenación con **tf.concat**.
- Apilación con **tf.stack**.
- Selección con **gather/gather_nd**.

Integración con autograd. Autograd es una herramienta para el cálculo automatizado de derivadas (gradientes).

Calcula la derivada de una función matemática (en un punto) directamente usando el grafo de cómputo y una cinta donde se almacenan las operaciones realizadas.

```
1 import tensorflow as tf
2
3 # Variable en un punto
4 X= tf.Variable(3.0)
5
6 # Funcion Y=f(X)
7 with tf.GradientTape() as tape:
8     Y=X**2
9
10 # Calcular y' = dy/dx
11 Derivada= tape.gradient(Y, X)
12
```

Conjuntos de datos en Tensorflow:

- Disponibilidad de tipos para representar **datasets** (**tf.data.Dataset**)
- Útiles con conjuntos de datos grandes (que no caben en memoria y hay que ir leyendo de disco parcialmente múltiples veces).
- Ejemplos:
 - **tf.data.Dataset.from_tensor_slices(x)**: Crea un conjunto de datos con los datos dados en el tensor x.
 - **tf.data.Dataset.zip((x,y))**: Crea un conjunto de datos de pares (x,y)
 - **dataset.shuffle()**: Mezcla los patrones del **dataset** de forma aleatoria.
 - **dataset.batch()**: Organiza el **dataset** en conjuntos disjuntos más pequeños (batches)
 - **etc., etc., etc.,**

Instalación de la utilidad: **pip install tensorflow_datasets**

Ejemplos básicos introductorios de Tensorflow:

Cuaderno **IntroTensorflow.ipynb**.

Copia el cuaderno en tu **GDrive** desde **PRADO**, y ábrelo con **Colab**.

Sigue las instrucciones para familiarizarte con la biblioteca.

Contenido

- 1 Redes Neuronales y Deep Learning
- 2 Tensorflow
- 3 Introducción a Keras**
- 4 Redes convolucionales
- 5 Autoencoders
- 6 Ejercicios y entrega

¿Qué es Keras?

Keras es una API de alto nivel para Deep Learning escrita en Python.

Es la API de alto nivel de **TensorFlow** para **Deep Learning**, aunque también ha sido utilizada en otros frameworks como Theano, CNTK, MXNet, etc.

Documentación: <https://keras.io/>

Código (GitHub): <https://github.com/keras-team/keras>

Las clases de **Keras** principales para el diseñador de modelos de Deep Learning son las siguientes:

- **tf.keras.Layer**: abstrae el concepto de **capa** de una red neuronal.
- **tf.keras.Model**: abstrae el concepto de modelo (red neuronal o módulo usable dentro de la misma).
- **tf.keras.Sequential**: permite la construcción rápida de modelos de redes neuronales hacia adelante (*feedforward*).

En **Keras** hay multitud de tipos de **capas** preconstruidas. Algunas de ellas:

<https://keras.io/api/layers/>

Core layers

- Input object
- Dense layer
- Activation layer
- Embedding layer
- Masking layer
- Lambda layer

Convolution layers

- Conv1D layer
- Conv2D layer
- Conv3D layer
- SeparableConv1D layer
- SeparableConv2D layer
- DepthwiseConv2D layer
- Conv1DTranspose layer
- Conv2DTranspose layer
- Conv3DTranspose layer

Pooling layers

- MaxPooling1D layer
- MaxPooling2D layer
- MaxPooling3D layer
- AveragePooling1D layer
- AveragePooling2D layer
- AveragePooling3D layer
- GlobalMaxPooling1D layer
- GlobalMaxPooling2D layer
- GlobalMaxPooling3D layer
- GlobalAveragePooling1D layer
- GlobalAveragePooling2D layer
- GlobalAveragePooling3D layer

Recurrent layers

- LSTM layer
- GRU layer
- SimpleRNN layer
- TimeDistributed layer
- Bidirectional layer
- ConvLSTM1D layer
- ConvLSTM2D layer
- ConvLSTM3D layer
- Base RNN layer

Preprocessing layers

- Text preprocessing
- Numerical features preprocessing layers
- Categorical features preprocessing layers
- Image preprocessing layers
- Image augmentation layers

Normalization layers

- BatchNormalization layer
- LayerNormalization layer
- UnitNormalization layer
- GroupNormalization layer

Regularization layers

- Dropout layer
- SpatialDropout1D layer
- SpatialDropout2D layer
- SpatialDropout3D layer
- GaussianDropout layer
- GaussianNoise layer
- ActivityRegularization layer
- AlphaDropout layer

Algunas capas:

- **Input**: Capas de entrada de datos.
- **Dense**: Capas densas (completamente conectadas)
- **Conv2D**: Capas convolucionales 2D.
- **MaxPooling1D/2D** / **AveragePooling1D/2D**: Capas de reducción/resumen de datos.
- **BatchNormalization**: Capa de normalización de datos en lotes.
- **Dropout**: Capa de regularización.
- **Flatten**: Transformación de capa anterior a 1D.
- **etc., etc., etc., ...**

<https://keras.io/api/layers/>

Algunas capas disponibles en `tf.keras.layers`:

- **Input**: `keras.Input( shape=None, batch_size=None, dtype=None, ...)`
- **Dense**: `Dense(units, activation = None, ...)`
- **Conv2D**: `Conv2D(filters, kernel_size, strides = (1,1), padding = 'valid', activation = None, ...)`
- **MaxPooling2D**:
`MaxPooling2D(pool_size = (2,2), strides = None, ...)`
- **BatchNormalization**: `BatchNormalization()`
- **Dropout**: `Dropout(rate, noise_shape = None, seed = None)`
- **Flatten**: `Flatten()`
- **etc., etc., etc., ...**

<https://keras.io/api/layers/>

Keras: Construcción de modelos

En **Keras** hay tres formas de definir redes neuronales o modelos (<https://keras.io/api/models/>):

- **Sequential** (https://keras.io/guides/sequential_model/) fuerza a que todas las capas de la red vayan una detrás de otra de forma secuencial, sin permitir ciclos ni saltos entre las capas.
- **Model o Functional** (https://keras.io/guides/functional_api/) permite cualquier tipo de red neuronal, incluyendo ciclos y saltos entre capas.
- **Model subclassing** (https://www.tensorflow.org/guide/keras/custom_layers_and_models) permite implementar cualquier cosa “*from scratch*”. Se usa si se tienen casos de uso complejos y muy particulares.

Keras: Construcción de modelos secuenciales

Se realiza mediante el tipo de modelo **Sequential**. Por ejemplo:

```
1 model = Sequential()  
2 model.add(Dense(50, input_dim=4, activation='relu'))  
3 model.add(Dense(12, activation='relu'))  
4 model.add(Dense(3, activation='softmax'))  
5
```

... crea un modelo donde:

- La primera capa oculta tiene 50 neuronas activadas con ReLU. Las entradas a la capa deben tener dimensión 4 componentes.
- La segunda capa oculta tiene 12 neuronas activadas con ReLU.
- La capa de salida tiene 3 neuronas activadas con SoftMax.
- Todas las capas son **densas** (completamente conectadas entre sí).

Construcción de modelos (no necesariamente) secuenciales

Se realiza mediante la concatenación de capas que hacen, a su vez, uso de otras capas como entrada (el tipo **Model** o **functional** descrito previamente). Por ejemplo:

```
1 input1 = Input(shape=(4,))
2 hidden1 = Dense(50, activation='relu')(input1)
3 hidden2 = Dense(12, activation='relu')(hidden1)
4 output = Dense(3, activation='softmax')(hidden2)
5 model = Model(inputs=input1, outputs=output)
6
```

... crea un modelo con la misma estructura que el de la diapositiva anterior.

Todos los modelos, independientemente de cómo se hayan creado, comparten las siguientes funcionalidades:

- Función **predict** para obtener las salidas del modelo para unas entradas.
- Función **compile** para optimizar el grafo de cómputo.
- Función **fit** para entrenar el modelo con un algoritmo de Keras.
- Función **call**: Implementa el comportamiento del modelo.
- Función **build**: Indica a TensorFlow cómo debe construirse el modelo.
- Función **summary**: Devuelve/muestra un resumen con la arquitectura del modelo.

Keras: Funciones de pérdida

La mayoría de las capas de Keras tienen sus propios parámetros a optimizar. El **entrenamiento** de un modelo conlleva la minimización del error cometido por el mismo con respecto a una **función de pérdida**. Keras tiene algunas implementadas en el paquete **tf.keras.losses**:

- **Clasificación binaria**: `binary_crossentropy`
- **Clasificación multiclase**: `categorical_crossentropy`
- **Regresión**: `mean_squared_error`
- **etc., etc., etc., ...**

Documentación sobre las funciones de pérdida disponibles:

<https://keras.io/api/losses/>

Keras: Algoritmos de aprendizaje

Los **algoritmos de aprendizaje** de **Keras** están incluidos dentro del paquete **tf.keras.optimizers**. Algunos de ellos son:

- **Stochastic Gradient Descent**: `SGD(learning_rate, momentum, ...)`
- **Adaptive Moment Estimation**: `Adam(learning_rate, ...)`
- **Root-mean squared propagation**: `RMSprop(learning_rate, ...)`
- **etc., etc., etc., ...**

Documentación sobre los algoritmos de aprendizaje (optimizadores):
<https://keras.io/optimizers/>

Keras: Algoritmos de aprendizaje

Keras también permite utilizar tasas de aprendizaje adaptativas (que varían con el tiempo) en los algoritmos de aprendizaje. Los **planificadores** de las tasas de aprendizaje de **Keras** están incluidos dentro del paquete **tf.keras.optimizers**. Algunos de ellos son:

- **Decaimiento exponencial:**

`ExponentialDecay(initial_learning_rate, decay_steps, decay_rate)`

- **Decaimiento polinomial:**

`initial_learning_rate, decay_steps, end_learning_rate, power, ...)`

- **Decaimiento por función coseno:**

`CosineDecay(initial_learning_rate, decay_steps, alpha, ...)`

- **etc., etc., etc., ...**

Documentación sobre los **schedulers**:

https://keras.io/api/optimizers/learning_rate_schedules/

Keras: Compilación y entrenamiento

En **Keras**, la **compilación** permite agrupar de forma optimizada el modelo, el algoritmo de aprendizaje y la función de pérdida.

Se lleva a cabo con el método **compile** de **Model**:



```
1 modelo.compile(loss=funcion_de_perdida ,  
2                 optimizer=optimizador , metrics=['accuracy'],  
3                 ...)
```

El argumento **metrics** permite especificar las métricas (<https://keras.io/api/metrics/>) que se calcularán a lo largo de las épocas de entrenamiento.

Keras: Compilación y entrenamiento

Cuando el modelo ya se encuentra compilado, podremos usar el método **fit** de **Model** para entrenarlo. Se puede indicar, entre otras cosas:

- **epochs**: el número de iteraciones (épocas) de entrenamiento.
- **batch_size**: el tamaño de cada batch a usar para entrenar.
- **validation_split**: Tanto por uno de los datos usados para validación.

Un ejemplo:

```
1 modelo.fit(x_train, y_train, batch_size=batch_size,  
2           epochs=50, verbose=1, validation_split=0.2)  
3
```


Keras: Compilación y entrenamiento

Cuando se entrena un modelo con `fit()`, **Keras guarda el estado del modelo por donde se ha quedado entrenando.**

Esto quiere decir que si volvemos a usar `fit()`, el entrenamiento seguirá por donde se ha quedado, y **no empezará desde el principio.**

Si vamos a usar varias veces `fit()` sobre el mismo modelo definido previamente, tenemos que restablecer los pesos de la red a como estaban antes del entrenamiento o debemos re-crearlo de nuevo. Esto se puede hacer guardando los pesos de la red antes del primer entrenamiento (y después de la compilación) usando:

```
1  # Obtener los pesos actuales de la red
2  weights = my_model.get_weights()
3
4  # Establecer los pesos de la red
5  my_model.set_weights(weights)
6
```



Keras: Prueba y validación

Hay dos funciones principales:

- **predict()**: se utiliza para calcular las salidas de un modelo, como por ejemplo `y_pred= modelo.predict(X)`
- **evaluate()**: Se utiliza para evaluar un modelo con respecto a una o varias métricas seleccionadas, como por ejemplo `scores = modelo.evaluate(X,Y)`

Enlaces:

https://keras.io/api/models/model_training_apis/#predict-method

https://keras.io/api/models/model_training_apis/#evaluate-method

Ejemplos básicos introductorios de Keras:

Cuaderno **IntroKeras.ipynb**.

Copia el cuaderno en tu **GDrive** desde **PRADO**, y ábrelo con **Colab**.

Sigue las instrucciones para familiarizarte con la biblioteca **Keras** a nivel introductorio.

Contenido

- 1 Redes Neuronales y Deep Learning
- 2 Tensorflow
- 3 Introducción a Keras
- 4 Redes convolucionales**
- 5 Autoencoders
- 6 Ejercicios y entrega

¿Qué es una red convolucional?



Concepto (Filtro convolucional)

Es un filtro que aplica una función de convolución sobre la entrada. El filtro convolucional puede ser de 1D, 2D o múltiples dimensiones.

Una **capa convolucional** dentro de una red neuronal es una capa que aplica filtros convolucionales a sus entradas.

- Convolución 1-D (discreta) Y sobre función X :

$$S(t) = (X * Y)(t) = \sum_a X(a)Y(t - a)$$

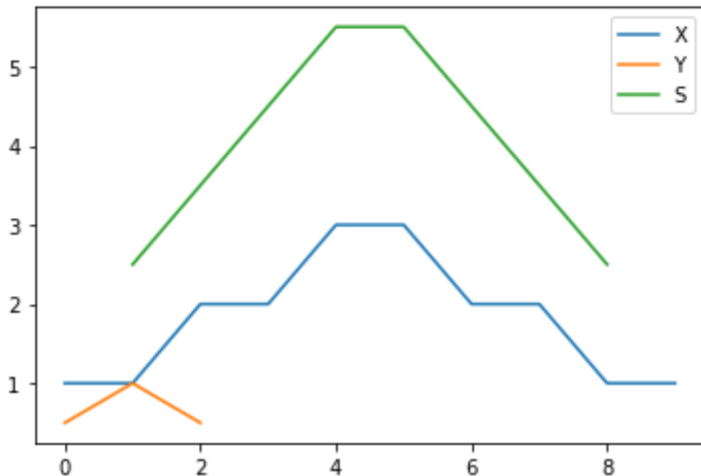
- Convolución 2-D Y sobre función X :

$$S(m, n) = (X * Y)(m, n) = \sum_j \sum_i X(i, j)Y(m - i, n - j)$$



¿Qué es una red convolucional?

Ejemplo de filtro convolucional 1-D $Y = (0,5, 1, 0,5)$ sobre los datos $X = (1, 1, 2, 2, 3, 3, 2, 2, 1, 1)$:



¿Qué es una red convolucional?

Ejemplo de filtro convolucional 2-D de tamaño 3x3:



Y

1	0	-1
2	0	-2
1	0	-1

Al filtro Y, con los valores dados, se le conoce como **filtro de Sobel horizontal** en tratamiento de imágenes.

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

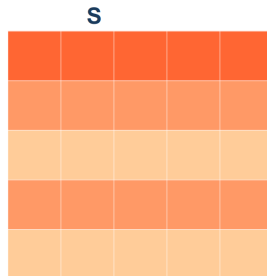
X



0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1



¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1				

¿Qué es una red convolucional?

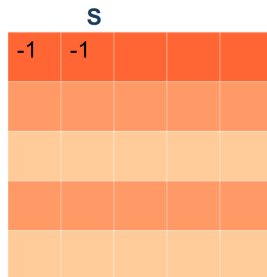
Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1



¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0		

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	1

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	1
-3				

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	1
-3	-3			

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	1
-3	-3	0		

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	1
-3	-3	0	3	

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	1
-3	-3	0	3	3

¿Qué es una red convolucional?

Ejemplo de aplicación de filtro de Sobel horizontal (convolución 2D):

X

0	0	0	0	0	0	0
0	0	0	0	0	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Y

1	0	-1
2	0	-2
1	0	-1

S

-1	-1	0	1	1
-3	-3	0	3	3
-4	-4	0	4	4
-3	-3	0	3	3
-1	-1	0	1	1

¿Qué es una red convolucional?

La implementación de una convolución 2-D es muy sencilla en Python:

```
# Convolución 2D
def conv2D(X, Y):

    Y_height, Y_width = Y.shape
    X_height, X_width = X.shape
    S_height = (X_height - Y_height)+1 #// stride + 1
    S_width = (X_width - Y_width)+1 #// stride + 1

    S= np.zeros((S_height, S_width), dtype=np.float64)

    for y in range(0, S_height):
        for x in range(0, S_width):
            S[y, x] = np.sum(X[y:y+Y_height, x:x + Y_width]*Y)
    return S
```

¿Qué es una red convolucional?

En el área de **Deep Learning**, una **convolución** se entiende como **Operación local lineal con una máscara** o **kernel**.

La **máscara** es el filtro.

Normalmente, los valores del filtro **son desconocidos** (parámetros **a aprender**).

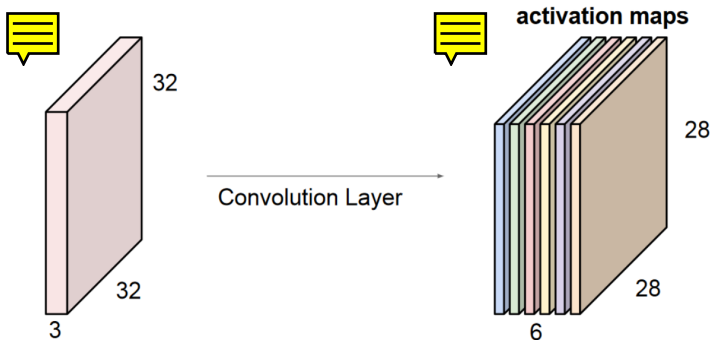
Primeros trabajos:

- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4), 541-551.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.

Redes convolucionales

Las capas convolucionales son especialmente útiles en áreas como el **tratamiento de imágenes** para extraer características de las mismas sobre las que aprender.

Podemos aprender tantos filtros (kernels) como deseemos. Por ejemplo, si tuviésemos imágenes de tamaño 32×32 y 3 niveles de color (profundidad), aprender 6 filtros de tamaño 5×5 nos devolvería como salida una “imagen” con profundidad 6 de tamaño 28×28 :



Ventajas:

- Menor número de parámetros frente a capas densas.
- Equivariante a traslaciones.
- Compartición de parámetros en toda la imagen.

Desventaja principal:

- Reducción de la señal de salida. El nuevo tamaño de la imagen de salida se reduce. Nuevo tamaño: $N_s = N_x - N_y + 1$ (N_x : Tamaño de señal de entrada; N_y : Tamaño de filtro; N_s : Tamaño de señal de salida).

Capas convolucionales 2D en Keras:

https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D

```
1 tf.keras.layers.Conv2D(  
2     filters,  
3     kernel_size,  
4     strides=(1, 1),  
5     padding='valid',  
6     activation=None,  
7     ...)  
8
```

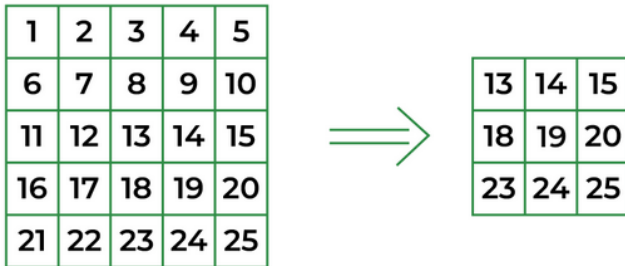
```
1 from tensorflow.keras.layers import Conv2D  
2  
3 # 32 filtros de 3x3, funcion de activacion relu  
4 conv_layer= Conv2D(32, kernel_size=(3, 3),  
5                     activation="relu")  
6
```


Hay dos formas principales de optimizar el rendimiento de una capa convolucional:

- Uso de **Padding**: ampliar los bordes de la imagen para que no se reduzca el tamaño. Dos posibilidades en **Keras**:
 - Rellenar con ceros (`padding='same'`) para obtener una imagen del mismo tamaño.
 - No aplicar padding (`padding='valid'`), opción por defecto.
- Uso de **strides**: En lugar de aplicar el filtro componente a componente, “saltamos” un número de componentes dado por el stride.

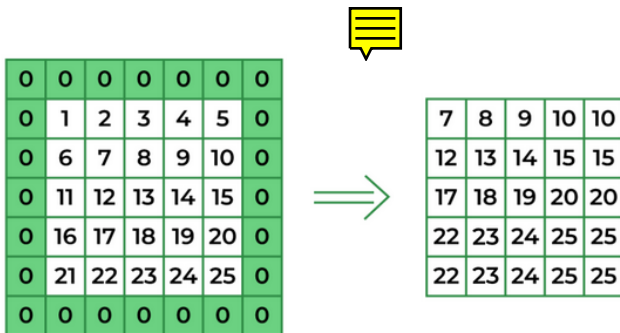
Redes convolucionales: Padding

Ejemplo de aplicación de filtro de 3x3 sobre imagen con padding='valid' (se reduce el tamaño de la salida):



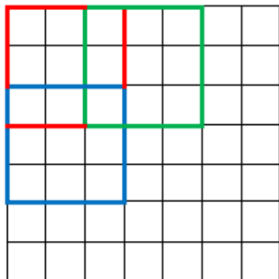
Redes convolucionales: Padding

Ejemplo de aplicación de filtro de 3x3 sobre imagen con `padding='same'` (la imagen de salida mantiene el tamaño de la original):



Redes convolucionales: Stride

Ejemplo de aplicación de filtro de 3x3 sobre imagen con `stride=(2,2)` (el filtro se mueve de dos en dos componentes verticales y dos en dos componentes horizontales):



Redes convolucionales: Integración con capas densas

En **Deep Learning**, lo normal es tener un modelo con capas de distinta naturaleza conectadas entre sí.

Las capas **convolucionales** requieren que su entrada tenga la misma dimensión que la estructura de la capa (1D, 2D, ...)

Las capas **densas** necesitan que sus entradas sean arrays.

Necesitamos incluir **capas de transformación** de la estructura de datos entre una **Conv2D** y una **Dense** y viceversa.

Redes convolucionales: Integración con capas densas

Para transformar la salida de una capa **Conv2D** a array: Capa **Flatten()**.

Para transformar la salida una capa **Dense** a matriz: Capa de cambio de forma **Reshape(target_shape)**

```
1 model= Sequential()  
2 model.add(Input(shape=(8,)))  
3 model.add(Dense(64))  
4 model.add(Reshape(target_shape=(8, 8, 1)))  
5 model.add(Conv2D(2, kernel_size=(3,3)))  
6 model.add(Flatten())  
7 model.add(Dense(2, activation='softmax'))  
8
```



Redes convolucionales: Capas de agregación

La **agregación** o **Pooling** se utiliza para reducir la dimensionalidad de los datos de salida de una capa convolucional, **extrayendo características** que puedan ser relevantes para capas futuras.

Es una técnica ampliamente utilizada en Deep Learning para reducir el tamaño de los datos procesados por una red (normalmente convolucional).

- **Average Pooling**: Reduce los datos calculando su media, dentro de una ventana de filtro indicada.

```
keras.layers.AveragePooling2D(pool_size, strides, padding, ...)
```

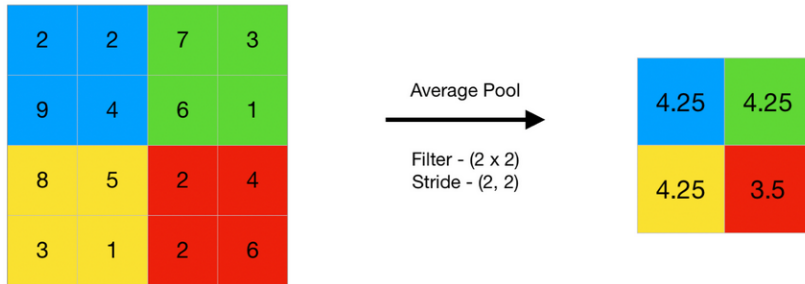


- **Max Pooling**: Reduce los datos calculando su máximo, dentro de una ventana de filtro indicada.

```
keras.layers.MaxPooling2D(pool_size, strides, padding, ...)
```

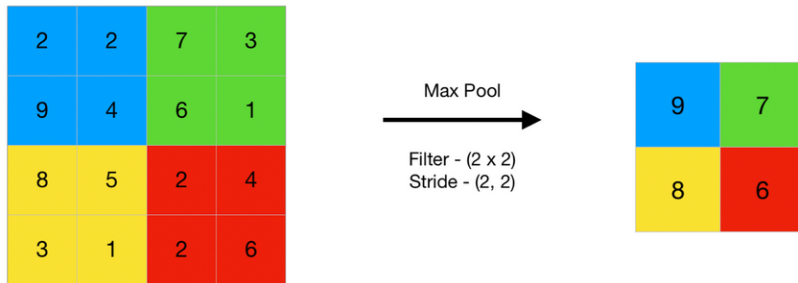

Redes convolucionales: Capas de agregación

Ejemplo de uso de `AveragePooling2D(pool_size=(2,2), stride=(2,2))`:



Redes convolucionales: Capas de agregación

Ejemplo de uso de `MaxPooling2D(pool_size=(2,2), stride=(2,2))`:



Normalización: Cuando las entradas a la capa tienen valores muy elevados (en valor absoluto), es necesario realizar una normalización de los mismos para evitar el cálculo de gradientes con valores también muy elevados.

Las capas de normalización en Keras:

https://keras.io/api/layers/normalization_layers/

Un ejemplo: Normalización por cada *batch* de datos que llega a la capa.

```
1 BatchNormalization(axis=-1, ...)
```

Normalización en Keras:

- **Normalización Batch**: normalización para cada subconjunto de entrada que tiene la red. Implementada en la capa `keras.layers.BatchNormalization`.
- **Normalización de Capa**: Normalización a valores $[-1, 1]$ dentro de los atributos de cada patrón de entrada. La diferencia con la normalización batch reside en que la Batch actúa sobre el conjunto de datos batch introducido, mientras que la de capa actúa sobre cada patrón de forma individual. Implementada en la capa `keras.layers.LayerNormalization`.
- **Normalización de unidad**: Normaliza cada patrón de entrada para que su norma sea igual a 1. Implementada en la capa `keras.layers.UnitNormalization`.

Redes profundas: regularización

Capas de regularización: Se utilizan para mejorar la calidad de la capacidad de generalización de la red.

Una posible estrategia consiste en **desactivar** algunas conexiones de la red durante el entrenamiento, de modo que el resto de conexiones se vean obligadas a cargar con la tarea de las conexiones no activas.

Esto fomenta la capacidad de generalización de las conexiones activas.

Las capas de regularización en Keras:

https://keras.io/api/layers/regularization_layers/

Un ejemplo: Regularización por **Dropout**.

```
1 keras.layers.Dropout(rate, noise_shape=None, seed=None,  
2 ...)
```

Ejemplos básicos de redes convolucionales:

Cuaderno **ConvNets.ipynb**.

Copia el cuaderno en tu **GDrive** desde **PRADO**, y ábrelo con **Colab**.

Sigue las instrucciones para familiarizarte con la biblioteca **Keras** y las redes convolucionales a nivel introductorio.

Contenido

- 1 Redes Neuronales y Deep Learning
- 2 Tensorflow
- 3 Introducción a Keras
- 4 Redes convolucionales
- 5 Autoencoders**
- 6 Ejercicios y entrega

Autoencoders

Los **autoencoders** se utilizan principalmente para **extracción de características** o elementos relevantes de los patrones de un conjunto de datos. Se consideran modelos de **Aprendizaje No Supervisado** dado que no requiere que las entradas al modelo se encuentren etiquetadas.

- **Aprendizaje no supervisado**: sólo requieren patrones de entrada.
- **Entrenamiento supervisado**: El mismo patrón de entrada es también salida.

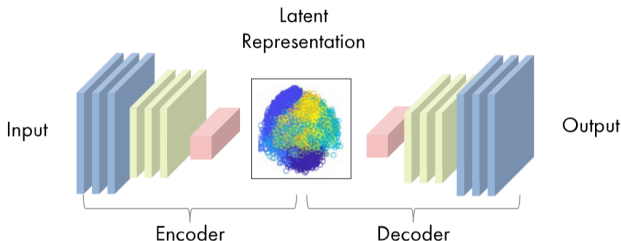
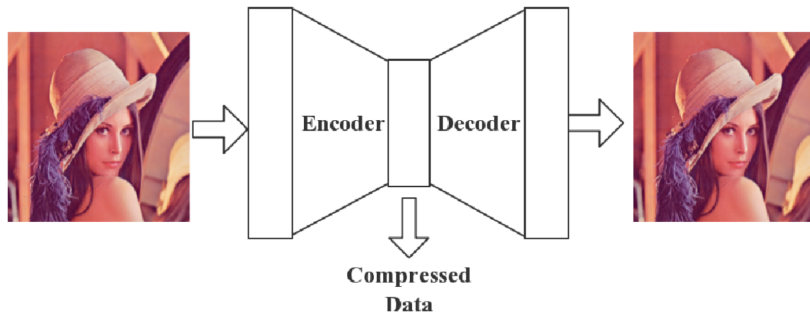


Figura 1: <https://la.mathworks.com/discovery/autoencoder.html>

Autoencoders

Objetivo de un autoencoder: Reproducir la entrada tan fielmente como sea posible, usando la mínima cantidad de datos posible.

- **Encoder:** Conjunto de capas que transforman la entrada a un conjunto de características (**embeddings**) en el **espacio latente** (espacio de características).
- **Decoder:** Conjunto de capas que tratan de reconstruir la entrada desde el espacio latente.



Autoencoders

La salida del módulo **encoder** se puede utilizar con otros fines. Por ejemplo: clasificación, regresión, reconstrucción de imágenes, etc.

La información del **espacio latente** nos proporciona una **representación abstracta** (o comprimida) de las características que hacen al dato de entrada ser como es.

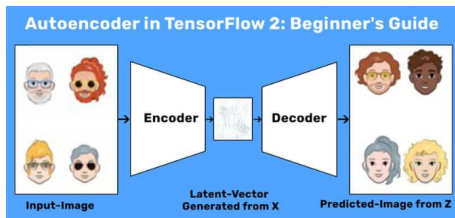


Figura 2: <https://learnopencv.com/autoencoder-in-tensorflow-2-beginners-guide/>

Contenido

- 1 Redes Neuronales y Deep Learning
- 2 Tensorflow
- 3 Introducción a Keras
- 4 Redes convolucionales
- 5 Autoencoders
- 6 Ejercicios y entrega**

Ejercicios y entrega de la práctica

El cuaderno **P3.ipynb** contiene instrucciones para llevar a cabo la práctica 3 de la asignatura.

Se debe entregar únicamente el cuaderno por PRADO. **IMPORTANTE:**

- Prestad atención al análisis y descripción de lo que se hace.
- Cuidad el preprocesamiento de datos.
- Seleccionad varios modelos y “jugad” con sus hiperparámetros.
- **Discusión** sobre los resultados obtenidos por cada modelo por separado, y **conclusiones** finales indicando si hay alguno mejor que otro o no, **justificando y analizando** el porqué.
- No entregar el análisis o una justificación de lo que se hace es como no entregar nada.

Problema 1: Clasificación (5 puntos).

- Contexto: Análisis de sentimientos.
- Uso de Inteligencia Artificial para poder identificar tipos de reseñas (positivas o negativas) con respecto a diferentes textos relativos a críticas de cine.
- Se trata de diseñar un modelo de Deep Learning que resuelva el problema, y hacer un análisis cuantitativo sobre la efectividad a la hora de clasificar una reseña.

¡No olvide activar el uso de GPUs de Google en el cuaderno!

Problema 2: Autoencoders (5 puntos).

- Contexto: Aprendizaje no supervisado con Deep Learning.
- Uso de Inteligencia Artificial para eliminar el ruido existente en imágenes aprendidas por una red neuronal de tipo **Autoencoder**.
- Se trata de diseñar un modelo de Deep Learning que aprenda las características de todos los dígitos del 0..9 (conjunto de datos MNIST), y usar los **embeddings** para eliminar el ruido existente sobre la imagen de un nuevo dígito, donde el ruido se ha introducido previamente de forma artificial.

¡No olvide activar el uso de GPUs de Google en el cuaderno!

Aprendizaje Automático

Introducción a Deep Learning

Manuel Pegalajar Cuéllar¹

¹Departamento de Ciencias de la Computación e Inteligencia Artificial. Universidad de Granada